

PACC: 대규모 연결 요소 계산

PACC: Large scale connected component computation on Hadoop and Spark

Ha-Myung Park, Namyong Park, Sung-Hyon Myaeng, U Kang (2020)

Partition-aware star 연산, edge filtering, sketching으로 분산 CC의 병목을 줄이는 방법

연결 요소 계산의 출력

각 정점은 도달 가능한 정점 중 가장 작은 ID를 component 대표자로 받는다

Connected component	최소 ID	출력 예
{5, 11}	5	5→5, 11→5
{3, 6, 12}	3	3→3, 6→3, 12→3
{1, 2, 4, 7, 8, 9, 10}	1	모든 정점→1
전제	정점 ID는 유일한 전체 순서	대표 ID가 component를 식별

PACC의 답은 임의 그룹 이름이 아니라 각 정점의 최소 도달 ID다

분산 CC의 세 병목

라운드 수, 중간 간선, reducer 편향은 서로 다른 비용이며 한 지표로 합칠 수 없다

병목	발생 원인	시스템 영향
많은 round	반복마다 일부 연결 정보만 전파	Job 준비와 HDFS I/O 반복
큰 intermediate	이웃 집합 또는 복제 edge 전달	Shuffle, disk, network 비용
Last reducer	최소 ID 정점에 edge 집중	소수 reducer가 전체 시간을 지배
PACC 목표	Partition, filtering, sketching	세 비용을 다른 기법으로 완화

빠른 CC 알고리즘은 round뿐 아니라 이동하는 edge와 reducer별 부하도 함께 줄여야 한다

Hash 계열과 alternating 비교

PACC 논문은 round 보장, 실무 속도, intermediate 크기, load balance의 tradeoff를 동기로 삼는다

방법	논문이 소개한 장점	논문이 지적한 한계
Hash-Greater-to-Min	$O(\log V)$ round 보장	원본의 2배를 넘는 intermediate
Hash-to-Min	관련연구에서 실무상 더 빠름	Log round 보장 없음, 큰 intermediate
Alternating	Round intermediate가 입력 이하	별 중심으로 edge가 몰림
Alt-opt	고차 정점을 복제해 분산	Edge 증가 가능, 편향 잔존

Hash-Greater-to-Min의 $O(\log |V|)$ 보장은 PACC의 수렴 정리가 아니다

Alternating의 last reducer 경로

Star 연산이 반복되면 많은 edge가 component의 최소 ID를 endpoint로 공유한다

01	초기 연결 그래프	Edge가 여러 key와 reducer에 분산
↓		
02	Star 연산 반복	큰 이웃과 작은 이웃을 minimum ID 쪽으로 재연결
↓		
03	별 그래프 수렴	Component의 많은 edge가 최소 ID를 공유
↓		
04	Reducer key=min	한 reducer가 거대한 neighbor list를 처리
↓		
05	전체 지연	다른 reducer가 끝나도 마지막 reducer를 기다림

평균 edge 수가 작아도 한 최소-ID key의 neighbor list가 크면 round가 끝나지 않는다

PACC 전체 파이프라인

PACC-base의 partitioning과 computation에 filtering과 sketching을 단계적으로 더한다

01	Input G	Undirected graph의 중복과 self-loop를 제거한 edge set
↓		
02	Sketching	Chunk LocalCC와 localization으로 초기 edge 수를 축소
↓		
03	Partitioning	PA-large와 PA-small을 반복해 partition-aware 구조 생성
↓		
04	Edge filtering	다음 round의 out과 누적 보관할 in, cc를 분리
↓		
05	CC-Computation	Partition별 edge-induced subgraph에서 LocalCC 실행

PACC-base는 두 핵심 단계, PACC-ef는 filtering 추가, PACC는 sketching까지 포함한다

Hash partition은 작업 라벨이다

$\xi(u)$ 는 connected component를 미리 자르지 않고 정점을 ρ 개 작업 그룹에 배정한다

항목	정의	오해 방지
$\xi(u)$	정점 u 의 hash partition	정점 ID의 크기 구간이 아님
ρ	Partition 수	Component 수가 아님
Cross-partition edge	연결성 보존에 사용 가능	Partition 경계가 물리적 단절이 아님
Final dispatch	Edge (u,v) , $u > v$ 를 $\xi(u)$ 에 보냄	Edge-induced subgraph는 겹칠 수 있음

Partition은 답의 component가 아니라 star 연산과 LocalCC의 부하를 나누는 기준이다

PA-star 연산의 기호

Global minimum과 partition별 local minimum을 구분하면 두 연산의 재배선이 보인다

기호	뜻	Node 7 예
$\Gamma^-(u)$	u보다 ID가 작은 이웃	{1, 2, 4}
$\Gamma^+(u)$	u보다 ID가 큰 이웃	{8, 9, 10}
$m(u)$	u와 모든 이웃의 최소 ID	$m(7)=1$
$m_i(u)$	Partition i 안의 local minimum	$m_1(7)=1, m_2(7)=2$
$\xi(u)$	u의 partition label	$\xi(7)=1$

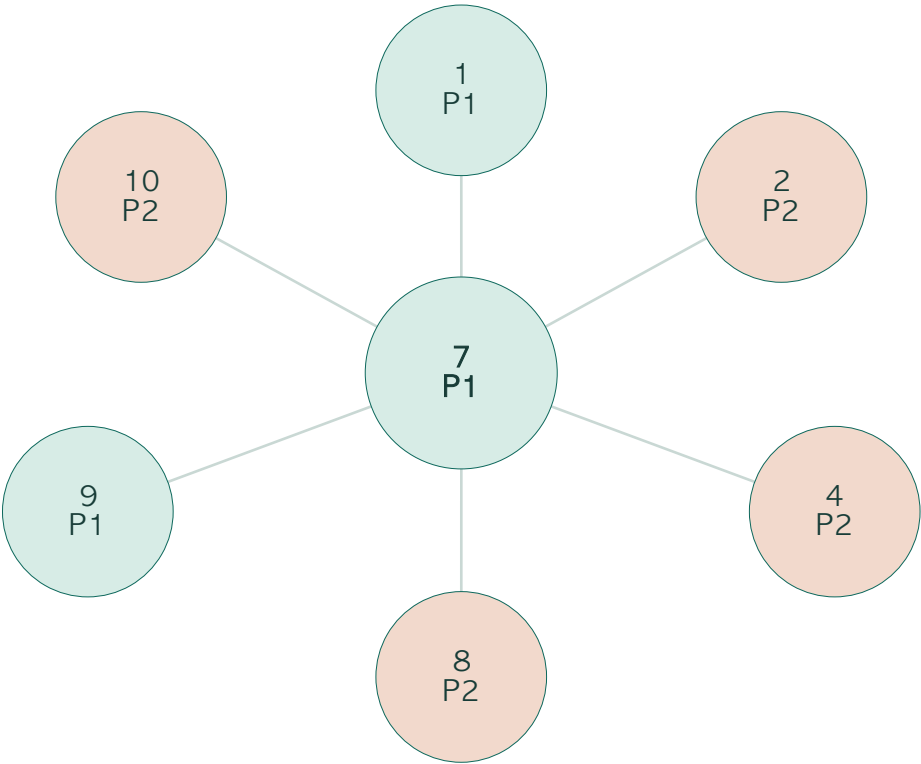
PA-star는 모든 이웃을 $m(u)$ 에 붙이지 않고 먼저 각 ξ 의 $m_i(u)$ 를 사용한다

Figure 4의 node 7 입력

Node 7의 local neighborhood만 나타내며 전체 graph G를 재현한 그림이 아니다

입력 이웃

출력 관계

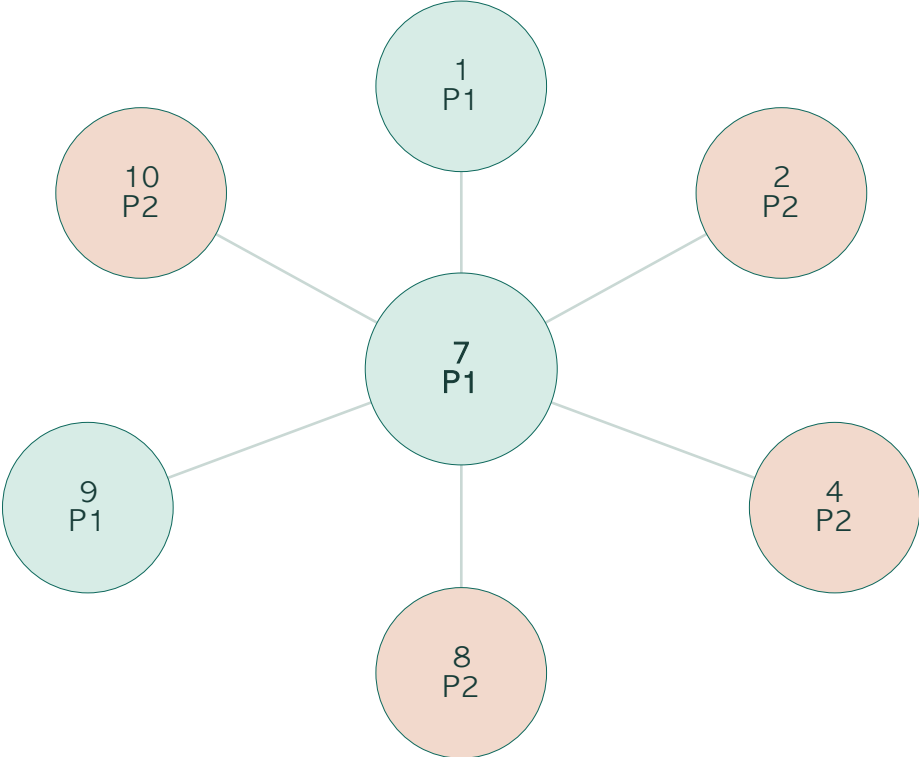


$$\begin{aligned} \rho &= 2; P1 = \{1, 7, 9\}, P2 = \{2, 4, 8, 10\} \\ \Gamma^-(7) &= \{1, 2, 4\}, \Gamma^+(7) = \{8, 9, 10\} \\ m(7) &= 1, m_1(7) = 1, m_2(7) = 2 \end{aligned}$$

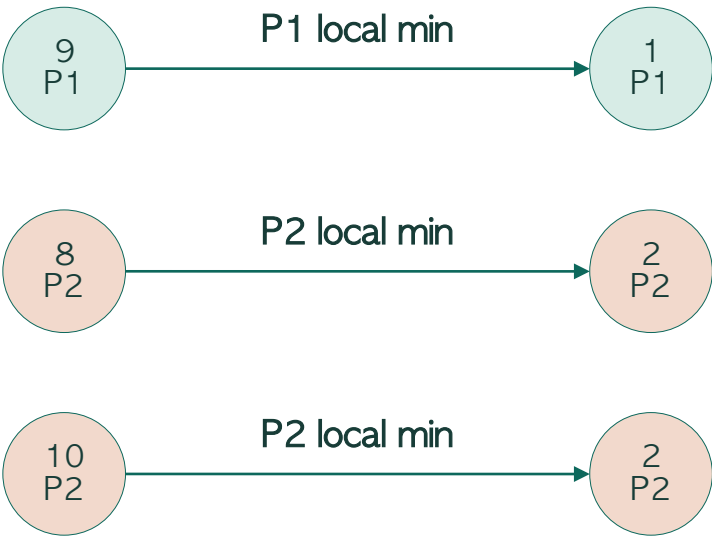
PA-large-star의 출력 관계

PA-large와 PA-small은 같은 Figure 4 입력을 따로 설명하며 연속된 graph 상태가 아니다

입력 이웃



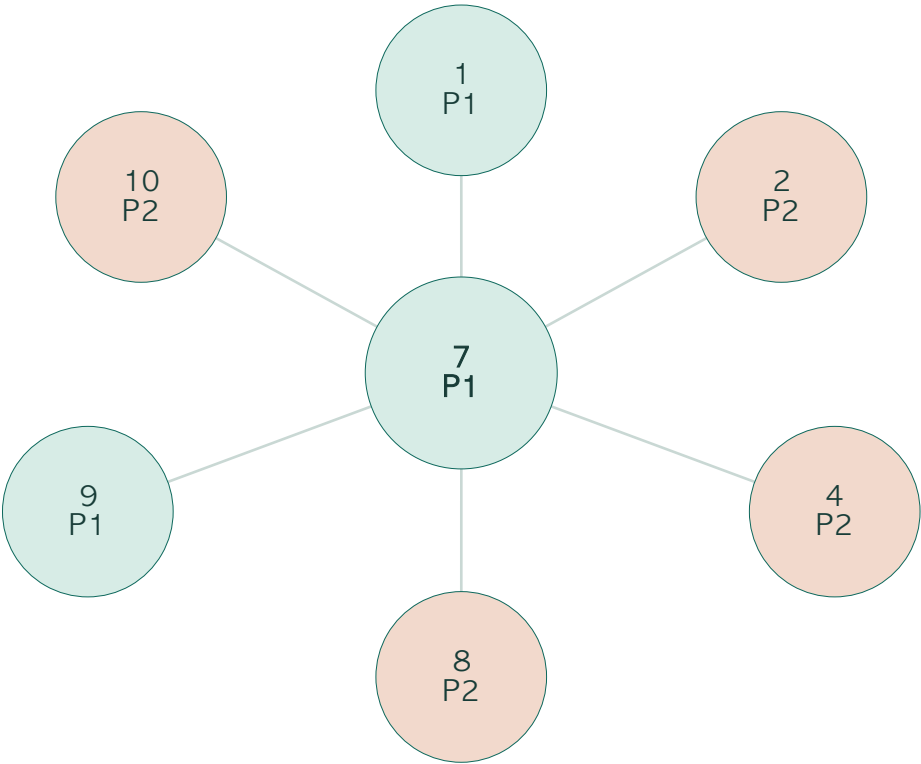
출력 관계



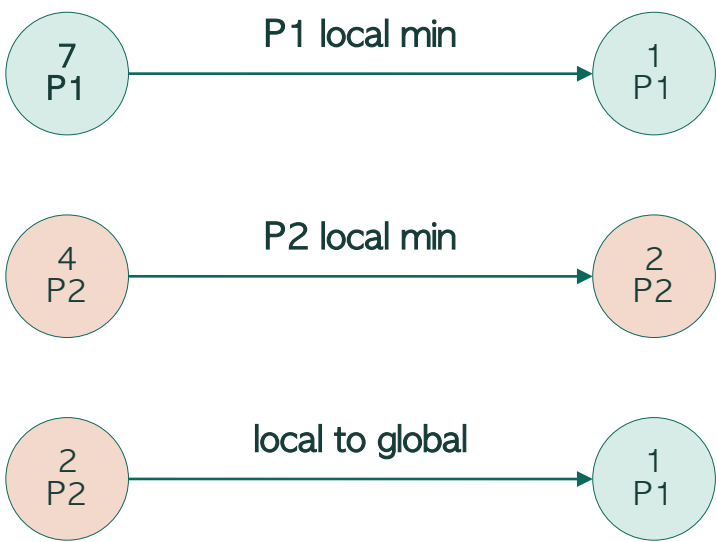
같은 Figure 4 입력
Large neighbors={8,9,10}
9→1, 8→2, 10→2

PA-small-star의 출력 관계

같은 입력에서 작은 이웃과 중심을 local minimum에 붙이고 local minimum을 global minimum에 잇는다
입력 이웃



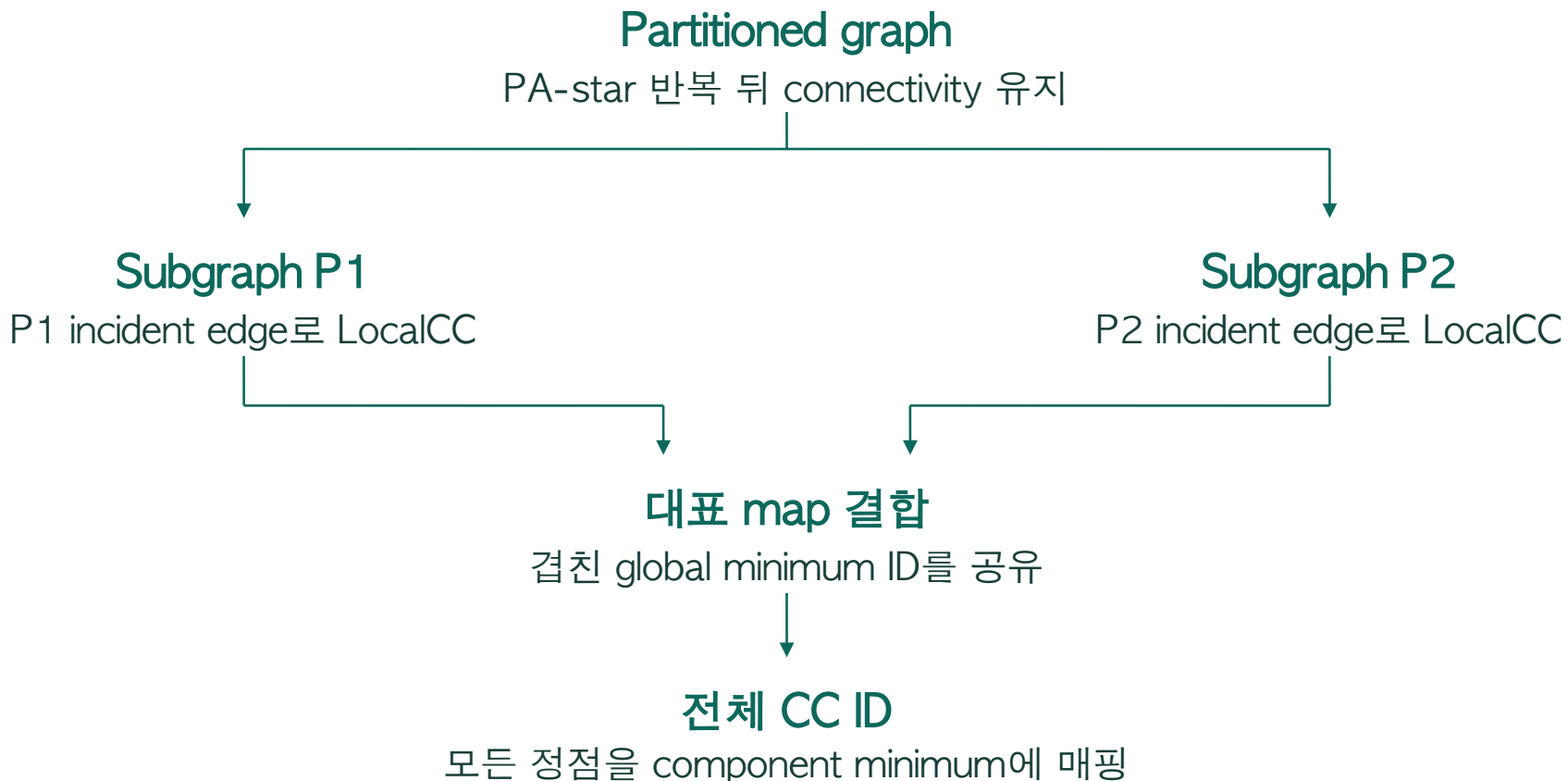
출력 관계



같은 Figure 4 입력
SmallUcenter={1,2,4,7}
7→1, 4→2, 2→1

Partition별 LocalCC가 가능한 이유

각 partition 정점은 local minimum에 연결되고 local minimum은 global minimum 경로를 가진다



PACC-base의 반복과 종료

두 partition-aware star 연산을 반복한 뒤 변경이 멈추면 CC-Computation을 한 번 수행한다

01	out = E	원래 undirected edge set으로 시작
↓		
02	PA-large-star	큰 이웃을 partition local minimum으로 재연결
↓		
03	PA-small-star	작은 이웃과 중심을 정리하고 local minimum을 연결
↓		
04	Convergence 검사	한 round에 edge 추가 또는 삭제가 없으면 반복 종료
↓		
05	CC-Computation	Partition별 edge-induced subgraph에서 LocalCC

PACC 논문은 종료 조건을 주지만 PACC 자체의 접근 round 상한은 제시하지 않는다

Edge filtering의 세 출력

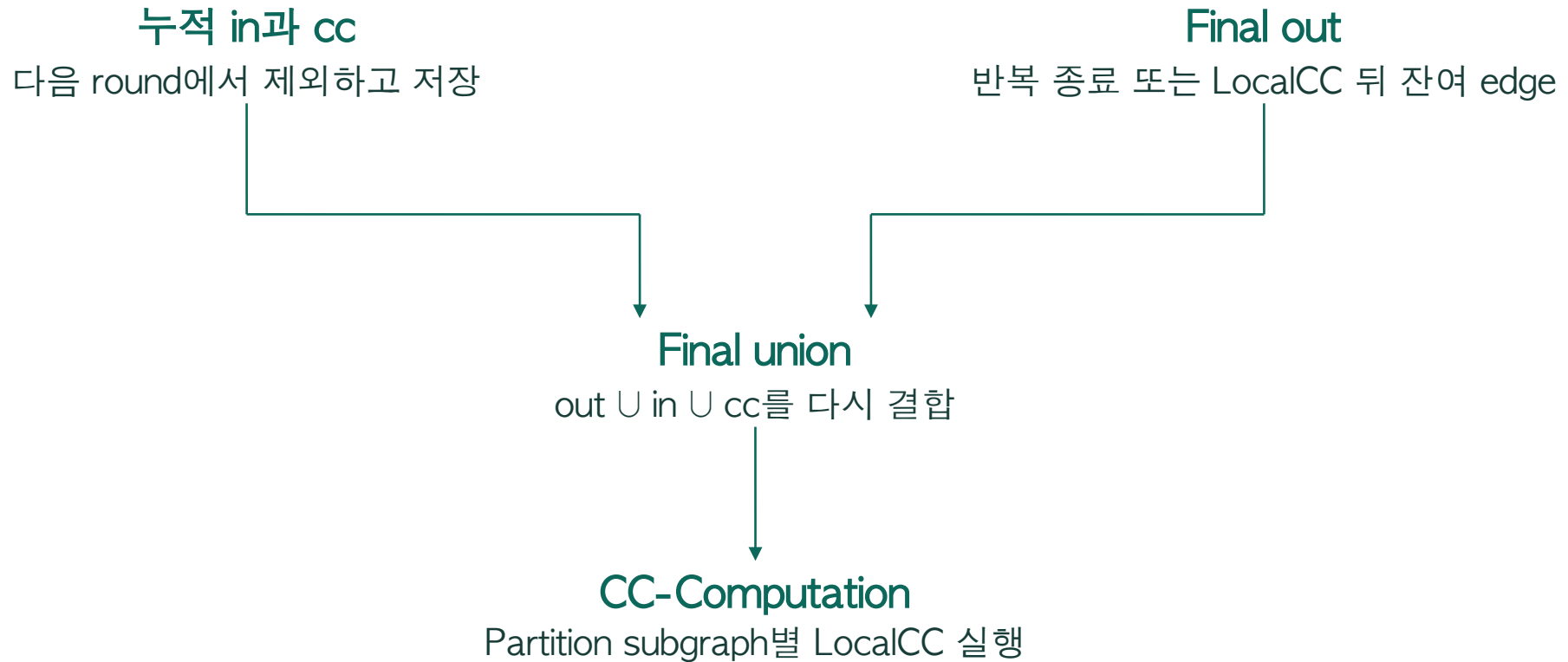
다음 round에 필요한 edge와 이미 정착한 edge를 분리해 반복 입력을 줄인다

Set	분류 조건과 의미	다음 사용
out	아직 star 연산이 바꿀 수 있는 edge	다음 분산 round 입력
in	같은 partition 안에서 독립 처리 가능한 forest edge	누적 보관 후 final union
cc	이미 완성된 minimum-center star edge	누적 보관 후 final union
Threshold τ	$\#out \leq \tau$ 이하	잔여 out에 LocalCC 실행

Filtering은 in과 cc를 답에서 삭제하지 않고 다음 반복에서만 제외한다

보관 edge와 잔여 edge의 합류

반복에서 빠진 in과 cc는 final out과 다시 합쳐져 CC-Computation의 정확성을 유지한다



Sketching 1 단계: chunk LocalCC

서로 겹치지 않는 storage chunk마다 연결 요소를 구해 대표자 edge만 남긴다

01	Edge-disjoint chunk	각 input edge를 정확히 하나의 E_i 에 저장
↓		
02	Chunk graph G_i	E_i 가 만드는 edge-induced subgraph 구성
↓		
03	LocalCC	각 chunk를 병렬 처리해 component minimum $r_i(u)$ 계산
↓		
04	대표자 edge	u 와 $r_i(u)$ 가 다를 때 $(u, r_i(u))$ 만 출력
↓		
05	Reduced graph G'	모든 chunk 출력의 union으로 connectivity 유지

첫 단계는 network 통신 없이 chunk별로 병렬 실행하지만 대표자에 edge가 몰릴 수 있다

Sketching 2단계: localization

대표자에 집중된 큰 이웃을 partition별 가장 작은 anchor를 경유하도록 재배선한다

01	Reduced graph G'	Chunk LocalCC 뒤 일부 대표자에 edge 집중
↓		
02	Partition별 anchor	u 의 큰 이웃 중 partition i 의 최소 $m_i^+(u)$ 선택
↓		
03	Anchor 유지	$m_i^+(u)$ 는 u 와 직접 연결
↓		
04	나머지 재배선	Edge (v,u) 를 $(v, m_i^+(u))$ 로 변경
↓		
05	Sketch graph G''	Connectivity를 유지하며 큰 neighbor list를 분산

Localization은 edge를 복제하지 않고 endpoint를 바꿔 연결성과 edge 수 상한을 지킨다

정확성 증명의 연결 고리

PACC는 graph 모양을 바꾸지만 connectivity와 representative 경로를 보존한다

01	PA-star 보존	Local minimum 경유 path로 원래 connectivity 유지
↓		
02	Sketch 보존	Chunk CC와 localization이 component를 바꾸지 않음
↓		
03	Filtered forest	in은 forest이고 final inUccUout도 forest
↓		
04	Minimum 연결	Local minimum마다 global minimum으로 가는 path 존재
↓		
05	독립 LocalCC	각 subgraph가 같은 component representative를 복원

정확성은 같은 edge를 유지해서가 아니라 같은 연결성과 minimum 경로를 유지해서 성립한다

간선 상한과 보장되지 않은 비용

Edge count 정리를 shuffle byte, peak RAM, 모든 reducer의 안전성으로 바꾸면 안 된다

원문 결과	정확한 범위	포함하지 않는 보장
PA-star edge 수 비증가	각 edge를 다른 edge로 변경	Map의 양방향 emit byte
Final edge $\leq V -1$	PACC-ef의 CC-Computation forest	각 round peak memory
$ E'' \leq \min(E , C V)$	Sketch output edge count	압축 byte와 sort 임시공간
Expected $O(V /\rho + \rho)$	Localization 뒤 큰 이웃 기대값	고확률 hash tail bound
Convergence 조건	Edge 추가와 삭제가 없을 때 종료	PACC의 점근 round 상한

논문 정리는 graph edge 수와 기대 부하를 다루며 실제 byte와 RAM은 구현 조건에 달려 있다

Hadoop와 Spark 구현 선택

논문의 scalability는 edge 상한과 함께 combiner, external sort, HDFS 보관에 의존한다

플랫폼	문제	PACC 구현 선택
Hadoop shuffle	Star Map이 edge를 양방향 emit	Combiner로 불필요한 이웃 감소
Duplicate edge	Cycle에서 같은 edge가 반복 출력	External-memory sort와 scan
Spark RDD	in과 cc를 끝까지 보유하면 memory 점유	out만 RDD, in과 cc는 HDFS
Spark grouping	groupByKey hash table은 대용량과 skew에 취약	partitionBy 뒤 external sort

PACC는 RAM을 없애지 않고 대형 grouping과 duplicate 처리를 외부 정렬로 넘긴다

실험 조건과 데이터 규모

성능 배수는 2020년 20-node HDD cluster와 Hadoop 2.7.3, Spark 2.2.0 조건의 결과다

범주	설정	해석
Cluster	20 machines, 각 6-core와 RAM 32 GB	각 machine에 4×2 TB HDD
Framework	Hadoop 2.7.3, Spark 2.2.0	Container memory 7 GB
PACC default	$\rho = 80, \tau = 20,000,000$	분할 수·전환 기준
Runtime	HDFS load와 result write 포함	End-to-end job time
ClueWeb12	6,257,706,595 V, 71,746,553,402 E	Self-loop와 duplicate 사전 제거

다른 cluster와 framework에서는 storage와 runtime 차이를 다시 측정해야 한다

세 기법의 실험 효과

Partitioning, filtering, sketching은 서로 다른 그래프 크기와 round 구간을 줄인다

기법	측정 대상	원문 결과	적용 범위
Partitioning	Reducer running-time 분포	매 iteration에서 best load balance	YahooWeb, Fig. 9
Edge filtering	Round별 input edge	수정 edge lower bound에 근접	YahooWeb, Fig. 10
Threshold τ	Twitter running time	$\tau = 2 \times 10^7$ 에서 최저	해당 설정, Fig. 11
Sketching	$ E / E'' $	Twitter 11.1, Friendster 1.7	Dataset별 차이, Fig. 12

Sketching reduction 11.1과 1.7은 dataset별 실험값이며 보편적인 압축 상수가 아니다

Hadoop과 Spark 결과의 분리

ClueWeb12 성공은 Hadoop 결과이며 Spark에서는 모든 비교 알고리즘이 실패해 결과를 생략했다

플랫폼	ClueWeb12	PACC 대 Alt-opt	추가 관찰
Hadoop	PACC와 PACC-ef만 성공	$2.9 \times - 10.7 \times$ faster	Friendster에서 YahooWeb 범위
Spark	모든 알고리즘 실패, 결과 생략	다른 graph에서 $2.3 \times - 8.8 \times$	YahooWeb에서 Alt와 PowerGraph OOM
작은 graph	해당 없음	Distributed overhead가 큼	PowerGraph 또는 Union-Find가 빠를 수 있음
Scale-out	Twitter machine 수 변화	PACC slope -0.74	Alt와 Alt-opt 약 -0.48

PACC는 20-node 실험에서 큰 graph에 잘 scale했지만 플랫폼별 최대 처리 규모는 달랐다

PACC 결과의 적용 한계

기대값, edge 수, 실험 speedup은 현재 hardware의 절대 memory와 runtime 보장이 아니다

한계	원문 범위	재사용 시 확인
2020 software stack	Hadoop 2.7.3, Spark 2.2.0	현재 scheduler와 storage 재측정
Hash 분포	Expected neighbor bound	Tail concentration 미명세
Edge count	Graph와 final forest 상한	Shuffle byte와 peak RAM 별도
Platform 차이	Hadoop만 ClueWeb12 성공	Spark 성공으로 일반화 금지
문제 범위	Undirected connected components	Triangle·최단 경로 자동 적용 불가

PACC는 CC의 minimum-root forest에 의존하며 다른 문제로 자동 확장되지 않는다

PACC의 핵심 흐름

Component를 미리 나누지 않고 connectivity를 유지한 채 계산 작업을 partition별로 나눈다

- | | | |
|----|---------------------|---|
| 01 | Connectivity 유지 | Star와 sketch 재배선이 원래 component를 보존 |
| ↓ | | |
| 02 | Local minimum 경유 | 이웃을 partition anchor에 붙여 global minimum 집중 완화 |
| ↓ | | |
| 03 | Settled edge 보관 | in과 cc를 반복에서 빼고 final union까지 유지 |
| ↓ | | |
| 04 | Forest와 subgraph | 각 partition이 global minimum path를 포함 |
| ↓ | | |
| 05 | Independent LocalCC | Subgraph별 결과를 같은 component ID로 복원 |

PACC partition은 결과 경계가 아니라 reducer 편향과 반복 data를 줄이는 작업 기준이다

공개 출처

논문 원문과 DOI, 저자가 공개한 구현 repository를 공개 경로로 확인할 수 있다

출처	용도	식별 정보
PLOS ONE	CC BY 공개 논문 원문	Article e0229936
Publisher DOI	정식 논문 식별자	10.1371/journal.pone.0229936
PACC GitHub	저자가 공개한 구현 참고	kmudmlab/PACC

알고리즘과 실험 주장은 PLOS 원문을 기준으로 하고 code는 구현 자료로 구분한다
